	Politechnika Śląska Katedra Grafiki Wizji Komputerowej i Systemów Cyfrowych	RAu6

Rok akademicki:	Rodzaj studiów*: SSI/NSI/NSM	Przedmiot (Języki Asemblerowe/SMiW):	Grupa	Sekcja
2025/2026	TI	SMIW	3	

Imię:	JAN	Prowadzący: OA/JP/KT/GD/GB/KH/AO	JP
Nazwisko:	LELKO		

Report

Temat projektu:

System monitorowania obecności człowieka

Główne założenia projektu:

Cel układu:
Bezkontaktowe monitorowanie obecności człowieka w wyznaczonej strefie. Urządzenie ma za zadanie analizę otoczenia i automatyczne reagowanie (np. generowanie alarmu) w przypadku zmiany statusu monitorowanego obiektu.
Zasilanie:
Układ będzie zasilany z zewnętrznego zasilacza sieciowego o napięciu 5V.

Data oddania: dd/mm/rrrr	09.09.2026

System monitorowania obecności człowieka

Raport końcowy projektu

Spis treści

1. Wprowadzenie **2**

2. Kontekst i analiza rynku **2**
2.1. Skala problemu . . . 2
2.2. Luka rynkowa . . . 2

3. Analiza techniczna **3**
3.1. Metoda detekcji . . . 3
3.2. Sensor radarowy . . . 3
3.3. Wariant implementacji radaru A121 . . . 3
3.4. Mikrokontroler . . . 4
3.5. Aktuatory, sygnalizacja, wejścia i zasilanie . . . 4
3.6. Schemat ideowy i prototyp . . . 5
3.7. Szacunkowy koszt prototypu . . . 6

4. Architektura systemu **7**
4.1. Protokół komunikacji z radarem (I^2C) . . . 7
4.2. Logika detekcji - cztery scenariusze . . . 8

5. Ewolucja projektu względem analizy **8**

6. Realizacja **9**
6.1. Konfiguracja z telefonu - captive portal . . . 9
6.2. Publikacja MQTT i inteligentny alarm . . . 10
6.3. Sygnalizacja i reset konfiguracji . . . 10
6.4. Instrukcja uruchomienia (quick start) . . . 10
6.5. Bezpieczeństwo . . . 11
6.6. Stan realizacji . . . 11

7. Ograniczenia i przypadki brzegowe **11**

8. Podsumowanie **11**

Bibliografia i źródła **13**

Streszczenie

Celem projektu było zaprojektowanie i zbudowanie działającego prototypu bezkontaktowego systemu monitorowania obecności człowieka, ukierunkowanego na bezpieczeństwo osób starszych mieszkających samotnie. System rozpoznaje cztery klasy zdarzeń: obecność, upadek, pożar oraz długotrwały bezruch. Rozpoznanie opiera się na danych z radaru fal milimetrowych 60 GHz, bez rejestracji obrazu. Dane są odczytywane po magistrali I^2C przez mikrokontroler ESP32-C6, który klasyfikuje sytuację lokalnie (*edge computing*) i publikuje status w formacie JSON do brokera MQTT. Przeglądarkowy panel opiekuna subskrybuje brokera przez WebSocket i pokazuje stan pomieszczeń w czasie rzeczywistym, wraz z alarmem dźwiękowym i rozróżnieniem stanu „pusty pokój" od „utrata łączności". Efektem prac jest kompletny, działający przepływ *end-to-end*: czujnik → firmware → broker → panel. Konfiguracja Wi-Fi i adresu brokera odbywa się z telefonu przez *captive portal*, bez zaszywania danych w kodzie.

1. Wprowadzenie

Starzenie się społeczeństwa czyni bezpieczeństwo samotnie mieszkających seniorów problemem o rosnącej skali. Upadek pozostawiony bez pomocy przez wiele godzin jest jedną z najczęstszych dróg do utraty samodzielności, a w skrajnych przypadkach zagraża życiu. Istniejące rozwiązania (kamery, opaski i przyciski SOS) mają istotne wady: naruszają prywatność, wymagają noszenia i ładowania albo są bezużyteczne, gdy poszkodowany jest nieprzytomny.

Celem projektu było opracowanie bezobsługowego, nieinwazyjnego systemu, który wykrywa obecność i zdarzenia zagrażające zdrowiu bez udziału użytkownika i bez rejestracji obrazu. Zakres prac objął: (1) analizę technologii detekcji, (2) rozpoznanie rynku i dobór komponentów, (3) projekt architektury sprzętowej i programowej, (4) implementację firmware realizującego lokalną klasyfikację zdarzeń oraz (5) budowę panelu opiekuna prezentującego status w czasie rzeczywistym.

Przyjęto, że jeden czujnik radarowy pod sufitem wystarczy do rozpoznania obecności, oddechu i upadku w pomieszczeniu, bez rejestracji obrazu i bez urządzeń noszonych przez użytkownika.

2. Kontekst i analiza rynku

2.1. Skala problemu

W Polsce mieszka ok. 10 mln osób w wieku 60+ (około 25% społeczeństwa), a coraz większa ich część żyje samotnie. Upadki są dominującą przyczyną hospitalizacji w tej grupie wiekowej. Według danych NCOA/CDC (USA) senior trafia na izbę przyjęć po upadku średnio co 11 sekund, a jedna na cztery starsze osoby upada w ciągu roku. Kluczowy jest czas reakcji. Pomoc udzielona szybko po upadku decyduje o dalszej samodzielności.

Źródła: GUS, „Sytuacja osób starszych w Polsce 2024", NCOA/CDC (USA).

2.2. Luka rynkowa

Trzy najpopularniejsze podejścia do monitorowania seniorów mają ograniczenia, które dyskwalifikują je w typowym środowisku domowym (tabela 1).

Odbiorcami rozwiązania są domy opieki, opieka domowa, szpitale oraz rodziny seniorów, co otwiera możliwość użycia modelu łączącego sprzedaż lub wynajem sprzętu (B2C) z licencjami placówek i panelem zbiorczym (B2B).

Tabela 1: Ograniczenia istniejących rozwiązań.

Rozwiązanie	**Główne ograniczenie**
Kamera	Narusza prywatność (łazienka, sypialnia), nie działa w ciemności.
Opaska / smartwatch	Trzeba pamiętać o noszeniu i regularnym ładowaniu.
Przycisk SOS	Bezużyteczny, gdy osoba jest nieprzytomna lub unieruchomiona po upadku.

3. Analiza techniczna

3.1. Metoda detekcji

Rozważono trzy technologie bezkontaktowej detekcji obecności (tabela 2). Radar fal milimetrowych jako jedyny zapewnia wystarczającą dokładność bez rejestracji obrazu.

Tabela 2: Porównanie metod detekcji.

Technologia	**Ocena**
PIR (pasywna podczerwień)	Najtańsza. Wykrywa tylko duże zmiany temperatury, nie widzi osób śpiących i nieruchomych. *Odrzucona.*
Kamera + analiza obrazu	Bardzo dokładna. Narusza prywatność i wymaga dużej mocy obliczeniowej. *Odrzucona.*
Radar fal milimetrowych	Przenika odzież i cienkie przeszkody, nie rejestruje obrazu, wykrywa mikroruchy (oddech). **Wybrana.**

3.2. Sensor radarowy

Porównano trzy moduły radarowe dostępne na rynku (tabela 3). Decydująca była zdolność detekcji mikroruchów (ruch klatki piersiowej - oddech), krytyczna dla funkcji medycznych.

Tabela 3: Porównanie modułów radarowych.

Moduł	**Pasmo**	**Detekcja mikroruchów i decyzja**
RCWL-0516	3 GHz	Bardzo słaba (tylko duży ruch). *Odrzucony* - zbyt prosty, brak danych do analizy.
HLK-LD2410	24 GHz	Średnia, mała precyzja. *Odrzucony* - za słaby do funkcji medycznych.
Acconeer A121	60 GHz	Bardzo dobra (ruch klatki piersiowej). **Wybrany** - najwyższa rozdzielczość, wykrywa mikroruch i makroruch.

3.3. Wariant implementacji radaru A121

Sam czujnik A121 wymagałby lutowania BGA i projektowania własnych anten, dlatego szukano gotowego modułu (tabela 4).

Gotowy moduł eliminuje projektowanie skomplikowanych obwodów i lutowanie BGA. Zintegrowane translatory napięć pozwalają podłączyć moduł bezpośrednio do pinu 5 V mikro-

kontrolera. Zachowana zostaje precyzja 60 GHz przy pominięciu kosztów oficjalnych zestawów ewaluacyjnych.

Tabela 4: Warianty implementacji radaru Acconeer A121.

Wariant	**Decyzja i uzasadnienie**
Acconeer A121 (goły czujnik)	*Odrzucony* - lutowanie BGA i własne anteny.
SparkFun XM125	*Odrzucony* - wysoka cena, brak dostępności.
Acconeer XE125 (zestaw ewaluacyjny)	*Odrzucony* - bardzo wysoka cena, rozbudowana platforma testowa.
Moduł Acconeer XM125	*Odrzucony* - rozwiązuje tor radiowy, ale wymaga własnej płyty bazowej i zasilania.
Waveshare A121 mmWave	**Wybrany** - zintegrowany STM32, antena i stabilizatory napięcia. Eliminuje lutowanie BGA i upraszcza zasilanie.

3.4. Mikrokontroler

Porównano serie układów ESP32 (tabela 5). Wybrano ESP32-C6 - jedyną serię łączącą energooszczędność z pełnym zestawem protokołów bezprzewodowych IoT.

Tabela 5: Porównanie serii ESP32.

Seria	**Rdzeń / łączność**	**Decyzja**
P-Series (P4)	RISC-V 2×400 MHz, bez radia	*Odrzucona*
S-Series (S3)	Xtensa 2×240 MHz, Wi-Fi 4 + BLE 5.0	*Odrzucona*
H-Series (H2)	RISC-V 96 MHz, 802.15.4 + BLE 5.3	*Odrzucona*
C-Series (C6)	RISC-V 160 MHz + rdzeń LP, Wi-Fi 6 + BLE 5.3 + Zigbee/Thread	**Wybrana**

Wi-Fi 6 (802.11ax) zapewnia stabilniejszą komunikację i lepsze zarządzanie energią. Koprocesor Low-Power (LP) nasłuchuje przerwań z radaru, gdy główny rdzeń śpi, minimalizując pobór prądu. Obsługa Zigbee/Thread otwiera przyszłą integrację z automatyką budynkową.

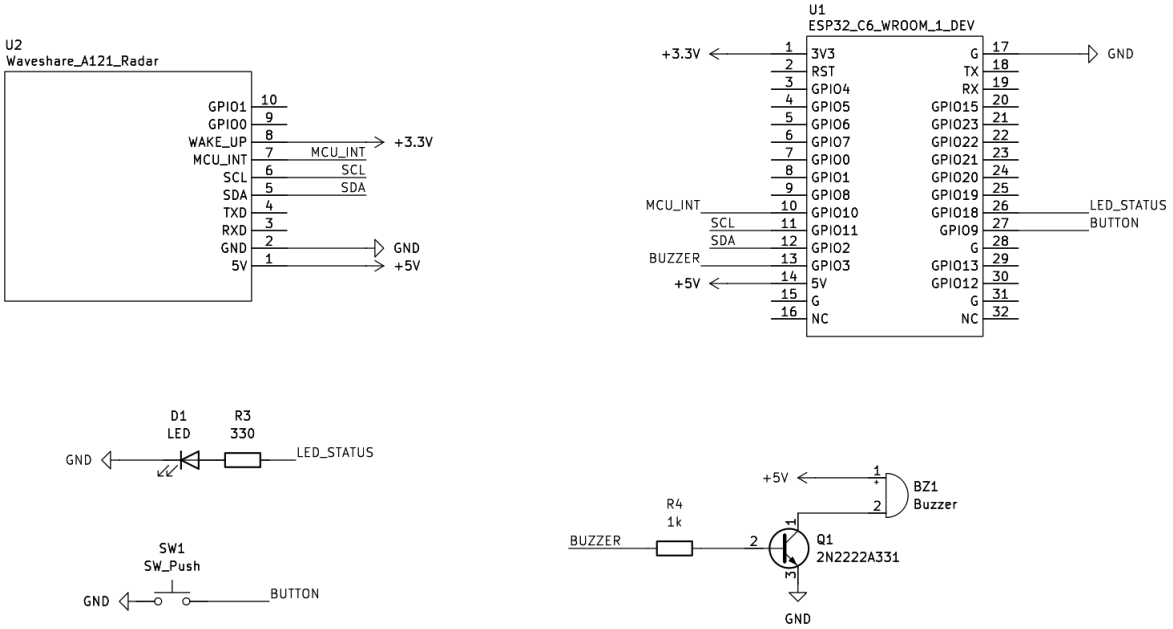
3.5. Aktuatory, sygnalizacja, wejścia i zasilanie

Warstwa lokalnej interakcji jest minimalna: buzzer wyzwala alarm dźwiękowy, dioda LED pełni rolę wskaźnika statusu, a przycisk służy do ręcznego kasowania alarmu i resetu konfiguracji. ESP32-C6 wykorzystuje sześć pinów GPIO: magistrala I^2C do radaru (SDA, SCL), linia `MCU_INT` radaru oraz buzzer, dioda i przycisk. Pin `WAKE_UP` radaru jest podciągnięty na stałe do 3,3 V, więc nie zajmuje GPIO. Zasilanie 5 V i wspólną masę rozprowadzono osobno.

Zasilanie: zewnętrzne 5 V z portu USB podłączone do modułu ESP32-C6-DevKit. Napięcie 5 V jest wyprowadzone z pinów ESP32 do modułu radaru. Oba moduły wykorzystują własne, wbudowane stabilizatory LDO, dzięki czemu główny schemat zasilania jest maksymalnie uproszczony.

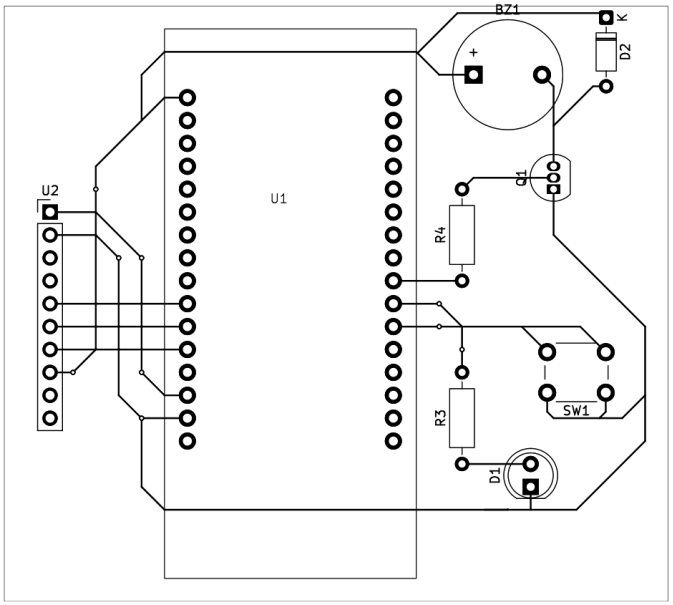
3.6. Schemat ideowy i prototyp

Warstwę sprzętową zaprojektowano w KiCad. Schemat ideowy (rys. 1) obejmuje moduł ESP32-C6-DevKit (U1), moduł radaru Waveshare A121 (U2), diodę sygnalizacyjną z rezystorem 330 Ω, przycisk oraz buzzer sterowany tranzystorem 2N2222 z rezystorem bazowym 1 kΩ. Przypisanie wyprowadzeń w firmware: `SDA` = GPIO2, `SCL` = GPIO11, `MCU_INT` = GPIO10, dioda = GPIO18, przycisk = GPIO9, buzzer = GPIO3. Pin `WAKE_UP` radaru jest zwarty na stałe do 3,3 V. Zaprojektowano też wstępne rozmieszczenie elementów na dedykowanej płytce PCB (rys. 2) pod docelową, zminiaturyzowaną wersję urządzenia. Zmontowany prototyp na płytce uniwersalnej pokazano na rys. 3.

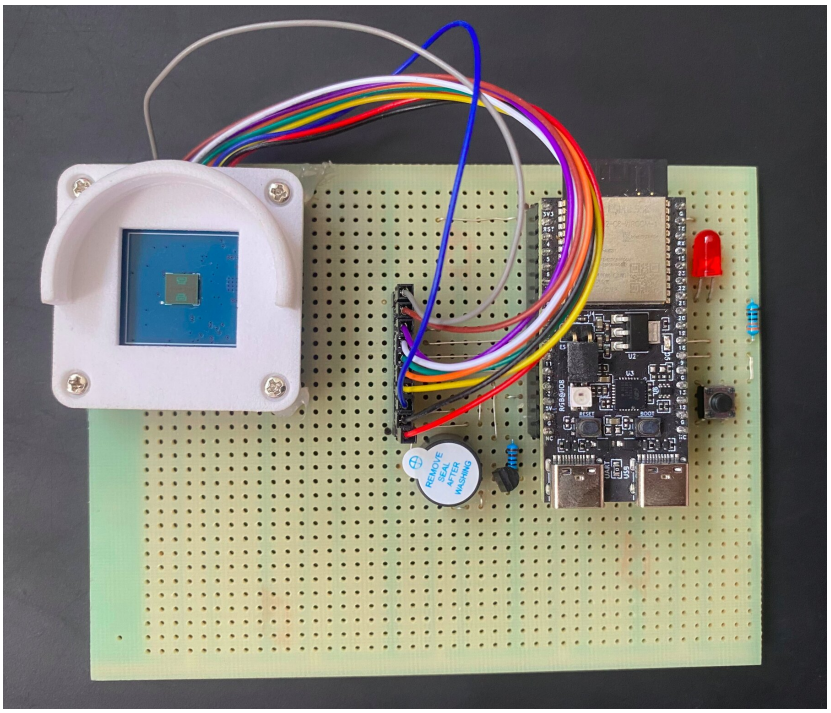


Rysunek 1: Schemat ideowy układu (KiCad). Wersja podstawowa. W prototypie diodę sygnalizacyjną uzupełniono dodatkową diodą RGB wbudowaną w moduł ESP32-C6.

Uwaga wdrożeniowa: krytyczny jest pin `WAKE_UP` podciągnięty do 3,3 V. Bez niego STM32 na module pozostaje w trybie Deep Sleep i odpowiada po I^2C samymi zerami, co łatwo pomylić z błędem okablowania.



Rysunek 2: Rozmieszczenie elementów i połączenia na dedykowanej płytce PCB.



Rysunek 3: Zmontowany prototyp urządzenia.

3.7. Szacunkowy koszt prototypu

Koszt budowy jednego egzemplarza zebrano w tabeli 6. Ceny są orientacyjne, z detalicznej sprzedaży modułów.

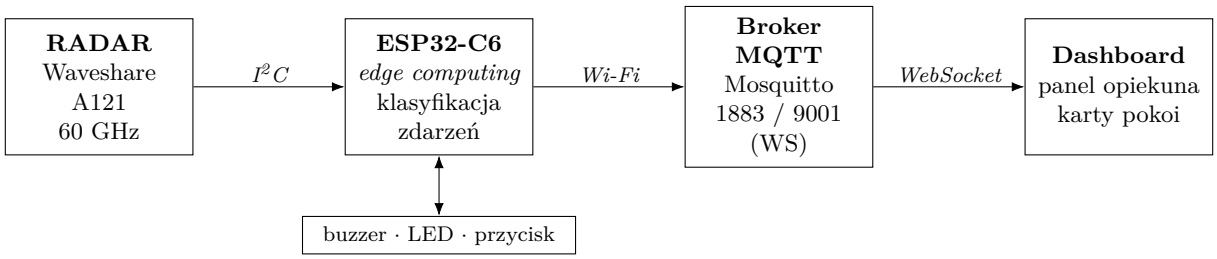
Tabela 6: Szacunkowy koszt prototypu.

Element	**Koszt**
Moduł radaru Waveshare A121 (A121 + STM32)	100 zł
ESP32-C6-DevKit	36 zł
Buzzer, dioda RGB, przycisk, rezystory	ok. 10 zł
Płytka uniwersalna, przewody, drobne elementy	ok. 10 zł
Razem	**ok. 155 zł**

Radar odpowiada za około dwie trzecie kosztu. Pozostałe elementy to tanie, powszechnie dostępne moduły, więc budowa kolejnych egzemplarzy oraz ewentualny serwis nie generują dużych kosztów. Jeden czujnik obsługuje jedno pomieszczenie, co pozwala planować koszt wdrożenia liniowo względem liczby monitorowanych pokoi.

4. Architektura systemu

System jest łańcuchem czterech ogniw: czujnik radarowy odczytywany po I^{2}C, mikrokontroler klasyfikujący zdarzenia lokalnie, broker MQTT oraz przeglądarkowy panel opiekuna (rys. 4). Kluczową decyzją architektoniczną jest lokalna klasyfikacja na urządzeniu. Reakcja następuje nawet bez połączenia z chmurą.



Rysunek 4: Schemat blokowy systemu: przepływ danych od czujnika radarowego do panelu opiekuna. Klasyfikacja zdarzeń odbywa się lokalnie na ESP32-C6.

4.1. Protokół komunikacji z radarem (I^{2}C)

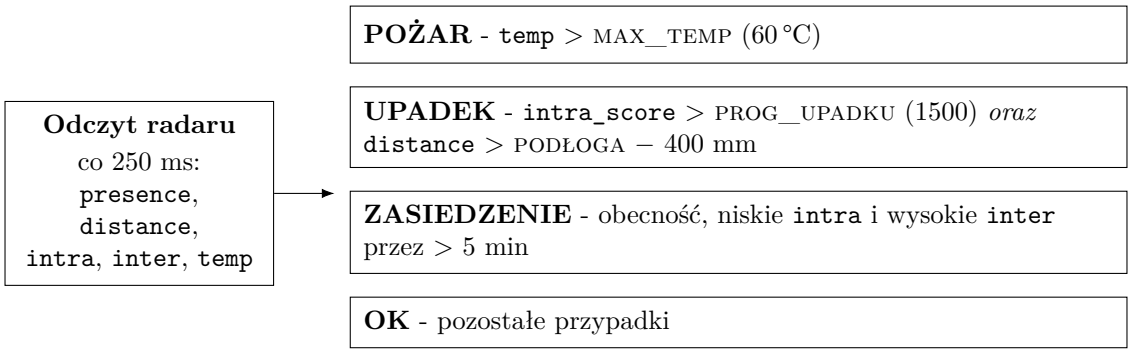
Moduł działa jako urządzenie I^{2}C slave pod adresem 0x52: adresy rejestrów są 16-bitowe (MSB first), a wartości 32-bitowe big-endian, przy czym każdy odczyt lub zapis obejmuje wielokrotność 4 bajtów. Po włączeniu zasilania radar nie mierzy, tylko czeka na komendy w rejestrze 0x0100: najpierw *apply config* (wartość 1), następnie *start* (wartość 2). Wyniki odczytuje się z rejestru 0x0010 jako cztery kolejne wartości 32-bitowe (tabela 7).

Tabela 7: Układ rejestru wynikowego (0x0010).

Bajty	**Pole**	**Znaczenie**
0-3	`presence_reg`	bit 0 = obecność. Górne 16 bitów = temperatura chipa
4-7	`distance_mm`	wykryta odległość [mm]
8-11	`intra_score`	energia ruchu szybkiego (mikroruch / dynamika)
12-15	`inter_score`	energia ruchu wolnego (makroruch / obecność statyczna)

4.2. Logika detekcji - cztery scenariusze

Klasyfikator uruchamiany co 250 ms przypisuje bieżącym odczytom jeden z czterech stanów (rys. 5).



Rysunek 5: Reguły klasyfikacji zdarzeń. Progi (PODŁOGA_MM, PROG_UPADKU, MAX_TEMP) są konfigurowalnymi stałymi w firmware.

Wynik klasyfikacji jest publikowany do brokera w formacie JSON, np.:

```
{"obecnosc":1, "odleglosc":2310, "mikro":1450,
"makro":220, "temp":41, "alarm":"OK"}
```

Publikacja jest dławiona do raz na 2 s, chyba że alarm jest inny niż `OK`. Wtedy wiadomość wysyłana jest natychmiast.

5. Ewolucja projektu względem analizy

Analiza projektu kończyła się na doborze komponentów i wstępnym schemacie blokowym. W toku implementacji założenia doprecyzowano oraz rozszerzono o warstwę programową. Najważniejsze zmiany zebrano poniżej.

- **Uproszczenie toru zasilania i komunikacji.** W analizie rozważany był wariant z gołym układem XM125, osobnymi stabilizatorami 3,3 V i 1,8 V oraz mostkiem USB-UART („pierwszy porzucony pomysł"). W wersji finalnej użyto gotowego modułu Waveshare A121 z własnymi stabilizatorami, podłączonego do ESP32-C6 bezpośrednio po magistrali I^2C. Odpadł osobny mostek UART i zewnętrzne przetwornice.
- **Wybór interfejsu radaru.** Analiza dopuszczała UART lub I^2C. W realizacji wybrano I^2C (adres 0x52, rejestry 16-bitowe, wartości 32-bitowe big-endian).
- **Sygnalizacja stanu.** Pojedynczą diodę LED zastąpiono adresowalną diodą RGB (WS2812) rozróżniającą kilka stanów: portal konfiguracji, łączenie z Wi-Fi, połączenie z brokerem, alarm oraz błąd.
- **Rozszerzona rola przycisku.** W analizie przycisk służył tylko do kasowania alarmu. W firmware krótkie naciśnięcie kasuje alarm, a przytrzymanie około 3 s czyści pamięć NVS i wraca do captive portalu.
- **Konfiguracja z telefonu.** Analiza nie obejmowała sposobu podawania danych sieci. Dodano captive portal (SoftAP, serwer DNS oraz HTTP) zapisujący SSID, hasło i adres brokera do NVS, bez rekompilacji firmware.
- **Logika detekcji i warstwa MQTT.** Analiza kończyła się na warstwie sprzętowej. Dodano klasyfikator uruchamiany co 250 ms (stany OK, POŻAR, UPADEK, ZASIEDZENIE), publikację JSON do brokera Mosquitto z dławieniem do raz na 2 s oraz przeglądarkowy panel opiekuna.

- **Przypisanie wyprowadzeń.** Zgrubny bilans „minimum 8 pinów” z analizy doprecyzowano. Faktycznie ESP32-C6 zajmuje sześć pinów GPIO: `SDA` = GPIO2, `SCL` = GPIO11, `MCU_INT` = GPIO10, buzzer = GPIO3, dioda = GPIO18, przycisk = GPIO9. `WAKE_UP` radaru idzie na stałe do 3,3 V, nie do GPIO.

6. Realizacja

Firmware napisano w środowisku ESP-IDF dla ESP32-C6. Kod dzieli się na warstwy: odczyt radaru po I^2C, pętlę klasyfikacji, klienta MQTT oraz obsługę konfiguracji sieci. Odpowiedzialności rozdzielono na osobne moduły (tabela 8).

Tabela 8: Moduły firmware.

Moduł	Odpowiedzialność
`main.c`	pętla odczytu radaru, logika alarmów, publikacja MQTT, obsługa buzzera i przycisku
`radar.c / .h`	warstwa I^2C radaru (inicjalizacja, odczyt rejestrów)
`wifi_mqtt.c / .h`	Wi-Fi (STA/SoftAP), NVS, klient MQTT, wybór trybu pracy
`captive_portal.c`	serwer HTTP portalu konfiguracji
`dns_server.c / .h`	DNS przekierowujący ruch na portal (192.168.4.1)
`status_led.c`	sygnalizacja RGB: portal / Wi-Fi / MQTT / alarm / błąd

6.1. Konfiguracja z telefonu - captive portal

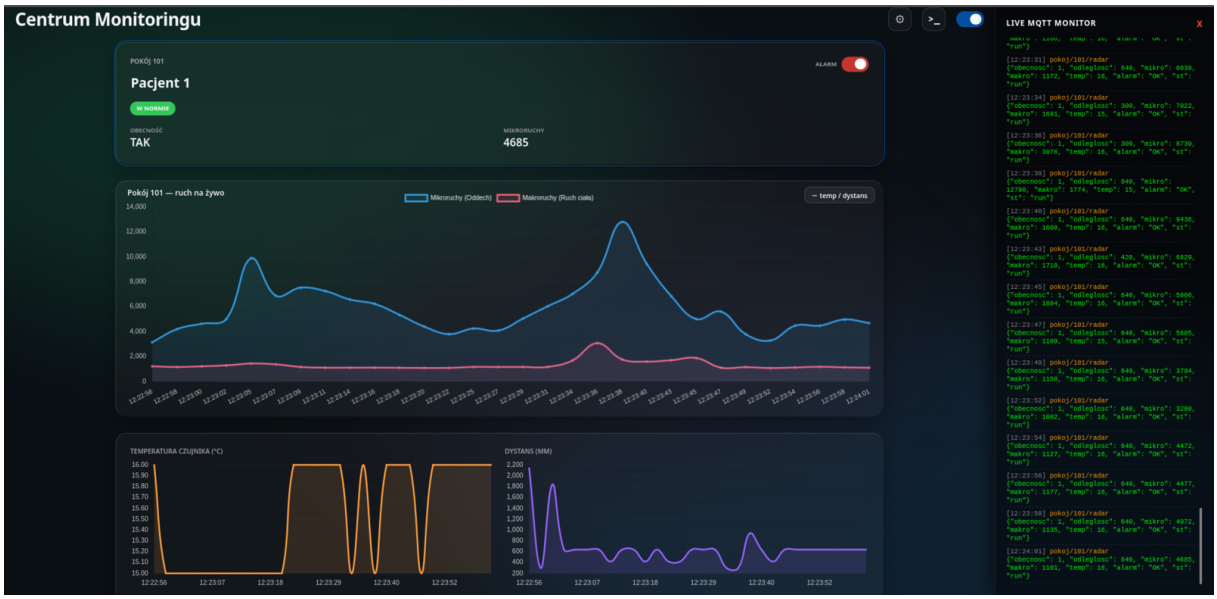
Dane sieci Wi-Fi i adres brokera MQTT nie są zaszyte w kodzie. Przy pierwszym uruchomieniu (lub po resecie) urządzenie stawia otwarty hotspot `Konfiguracja_Radaru` (SoftAP, 192.168.4.1). Wbudowany serwer DNS kieruje cały ruch na portal, więc na większości telefonów strona konfiguracji otwiera się sama. Użytkownik wpisuje SSID, hasło i adres brokera (rys. 6). Dane trafiają do pamięci NVS, a urządzenie restartuje się w tryb klienta (STA) i zaczyna publikować.



Rysunek 6: Portal konfiguracyjny na telefonie: SSID, hasło Wi-Fi oraz adres brokera MQTT (SoftAP `Konfiguracja_Radaru`, 192.168.4.1).

6.2. Publikacja MQTT i inteligentny alarm

W trybie STA firmware łączy się z brokerem i publikuje status pomieszczenia na temacie `pokoj/101/radar` (numer pokoju jest na razie zaszyty w kodzie). Statyczny dashboard łączy się z tym samym brokerem przez MQTT-over-WebSockets prosto z przeglądarki i pokazuje karty pokoi ze statusem w czasie rzeczywistym (rys. 7). Przełącznik *Alarm* uzbraja dozór: po uzbrojeniu panel reaguje dźwiękiem (Web Audio, bez plików) i czerwonym banerem, gdy nikogo nie ma pod czujnikiem albo gdy znika łączność. Kluczowe jest rozróżnienie stanu „pusty pokój" od „utrata łączności": watchdog wyszarza kartę i oznacza ją jako OFFLINE, gdy brak danych trwa dłużej niż 10 s, żeby cisza czujnika nie udawała spokoju.



Rysunek 7: Panel opiekuna: widok pokoju z przebiegami mikroruchu i makroruchu, temperaturą i dystansem oraz podglądem wiadomości MQTT na żywo.

6.3. Sygnalizacja i reset konfiguracji

Dioda RGB sygnalizuje kolejne stany urządzenia: tryb portalu konfiguracji, łączenie z Wi-Fi, połączenie z brokerem MQTT, alarm oraz błąd. Reset konfiguracji następuje po około 3 s przytrzymania przycisku (potrójny sygnał LED i buzzera). Urządzenie czyści NVS i wraca do captive portalu, bez potrzeby ponownego wgrywania firmware. Krótkie naciśnięcie przycisku kasuje trwający alarm dźwiękowy.

6.4. Instrukcja uruchomienia (quick start)

Z perspektywy użytkownika końcowego uruchomienie sprowadza się do kilku kroków, bez podłączania do komputera:

1. Zamontować czujnik pod sufitem i podłączyć zasilanie 5 V (USB).
2. Na telefonie połączyć się z otwartą siecią `Konfiguracja_Radaru`. Strona konfiguracji otworzy się sama.
3. Wpisać nazwę i hasło docelowej sieci Wi-Fi oraz adres brokera MQTT i zatwierdzić. Urządzenie zrestartuje się i zacznie publikować.
4. Stan odczytać z diody RGB: konfiguracja / łączenie z Wi-Fi / połączenie z brokerem / praca / alarm / błąd.
5. W razie potrzeby zmiany ustawień przytrzymać przycisk około 3 s (potrójny sygnał). Urządzenie wróci do kroku 2.

6.5. Bezpieczeństwo

System opiera się na ciągłej komunikacji bezprzewodowej, więc warto wprost opisać stan zabezpieczeń. W obecnym prototypie przyjęto świadome uproszczenia: dane sieci Wi-Fi i adres brokera są zapisywane w pamięci EEPROM bez szyfrowania. Broker Mosquitto pracuje w trybie anonimowym, bez uwierzytelniania i bez TLS. Ruch MQTT (1883) oraz MQTT-over-WebSockets (9001) jest nieszyfrowany. AP konfiguracyjny jest otwarty, a dashboard nie wymaga logowania. Jest to akceptowalne w izolowanej sieci laboratoryjnej, ale nie w docelowym wdrożeniu.

6.6. Stan realizacji

Projekt osiągnął stan działającego prototypu *end-to-end*: czujnik, firmware i dashboard tworzą kompletny przepływ. Cztery typy detekcji (obecność, upadek, przegrzanie, bezruch) działają, konfiguracja odbywa się z telefonu przez captive portal, a alarm wykrywa również utratę łączności z czujnikiem.

7. Ograniczenia i przypadki brzegowe

Prototyp świadomie nie był poddany pełnej walidacji skuteczności. Poniżej zebrano znane ograniczenia metody i bieżącej implementacji:

- **Zwierzęta domowe.** Radar reaguje na ruch i mikroruch, a nie na sylwetkę. Większy pies lub kot w polu widzenia może wywołać fałszywą obecność. Łagodzi to montaż pod sufitem (zwierzę jest dalej od czujnika, słabszy sygnał) oraz progi `intra`/`inter`.
- **Ruch firan i przeciągi.** Poruszana firana przy otwartym oknie albo nawiew z grzejnika czy wentylacji to drobny, powtarzalny ruch, który może podnosić `intra_score` i dać fałszywy sygnał ruchu. Wymaga kalibracji progów lub maskowania stref odległości.
- **Zasięg detekcji oddechu.** Mikroruch klatki piersiowej jest wiarygodny do około 2-3 m od czujnika. Dalej energia sygnału tonie w szumie i wykrywanie samego oddechu (bez większego ruchu) staje się zawodne.
- **Jeden czujnik = jedno pomieszczenie.** Układ nie liczy osób i nie rozróżnia ich tożsamości. Ściany działowe tłumią pasmo 60 GHz, co jest zaletą (brak przesłuchów między pokojami), ale wymusza po jednym czujniku na pomieszczenie.
- **Odbicia.** Duże metalowe powierzchnie i lustra w polu widzenia mogą dawać odbicia i zawyżać liczbę wykryć.

8. Podsumowanie

Zbudowany prototyp potwierdził trafność kluczowych decyzji projektowych: radar 60 GHz zapewnia detekcję mikroruchów bez naruszania prywatności, a lokalna klasyfikacja na ESP32-C6 daje reakcję niezależną od chmury. Dalszy rozwój koncentruje się na odporności łączności, dystrybucji panelu i bezpieczeństwie (tabela 9).

Tabela 9: Kierunki dalszego rozwoju.

#	Kierunek	Opis
1	Powiadomienia	Push / SMS bezpośrednio do opiekuna, niezależnie od panelu przeglądarkowego.
2	Panel z urządzenia	Serwowanie dashboardu prosto z ESP32 (serwer HTTP w trybie STA), bez zewnętrznego brokera.
3	Szyfrowanie transmisji	TLS dla MQTT (`mqtts`, port 8883) z weryfikacją certyfikatu brokera. WebSocket przez `wss`.
4	Uwierzytelnianie MQTT	Login i hasło (lub certyfikat) per urządzenie, ograniczenie tematów ACL do własnego pokoju.
5	Ochrona danych na urządzeniu	Włączenie flash encryption ESP32, aby poświadczenia Wi-Fi w NVS nie były czytelne po wyjęciu kości.
6	Zabezpieczenie konfiguracji	Hasło WPA2 na AP konfiguracyjnym, licznik prób, logowanie do dashboardu.
7	Placówki i pilotaż	Panel wielopokojowy dla domów opieki i szpitali oraz pilotaż w warunkach domowych.

Cel projektu, nieinwazyjny system detekcji obecności i upadków dla seniorów, został zrealizowany w postaci działającego prototypu. Architektura oparta na tanim ESP32-C6, jednym czujniku na pomieszczenie i otwartym standardzie MQTT jest łatwa do skalowania i integracji, co czyni rozwiązanie realnym punktem wyjścia do wdrożenia produkcyjnego.

Najważniejsze wnioski z realizacji:

- **Pin `WAKE_UP`.** Bez podciągnięcia tego pinu do 3,3 V radar odpowiada po I^2C samymi zerami, co łatwo pomylić z błędem okablowania. Był to najbardziej czasochłonny problem bring-upu.
- **Klasyfikacja na urządzeniu.** Przeniesienie logiki detekcji na ESP32-C6 daje reakcję nawet przy braku łączności z brokerem.
- **MQTT-over-WebSocket.** Panel w przeglądarce wymaga osobnego listenera WebSocket na brokerze. Bez portu 9001 dashboard się nie łączy.
- **Dławienie publikacji.** Ograniczenie do raz na 2 s odciąża sieć, a natychmiastowa wysyłka przy alarmie zachowuje krótki czas reakcji.
- **Captive portal.** Konfiguracja z telefonu wyraźnie skraca wdrożenie użytkownika końcowego.

Bibliografia i źródła

Dokumentacja techniczna

- Waveshare A121 Range Sensor - `waveshare.com/wiki/A121_Range_Sensor`
- Acconeer Exploration Tool - `docs.acconeer.com`
- Datasheet ESP32-C6-WROOM-1 - `documentation.espressif.com`
- ESP-IDF I^2C API Reference - `docs.espressif.com/projects/esp-idf`

Oprogramowanie i biblioteki

- ESP-IDF (Espressif), środowisko firmware dla ESP32-C6. Wykorzystane komponenty: `esp_wifi`, `mqtt_client`, `nvs_flash`, `esp_http_server`, `driver/i2c`, `led_strip`. `https://github.com/espressif/esp-idf`
- Eclipse Mosquitto, broker MQTT (nasłuch na portach 1883 oraz 9001 dla WebSocket). `https://mosquitto.org`
- MQTT.js, klient MQTT-over-WebSocket w panelu opiekuna. `https://github.com/mqttjs/MQTT.js`
- Chart.js, wykresy przebiegów mikroruchu i makroruchu na dashboardzie. `https://www.chartjs.org`

Dane rynkowe i statystyczne

- GUS, „Sytuacja osób starszych w Polsce 2024".
- NCOA / CDC (USA), statystyki upadków osób starszych.